

UGV Optimal Path Navigation Using A* Search

1. Title

Design and Implementation of an Optimal Path-Finding Algorithm for an Unmanned Ground Vehicle (UGV) in a Battlefield Grid

2. Problem Statement

An **Unmanned Ground Vehicle (UGV)** is a ground-based robot that can move from one location to another without direct human control.

In this problem, the UGV operates in a **70 × 70 km battlefield area**. The area is represented as a grid where:

- Each cell represents approximately **1 km × 1 km**.
- Some cells contain obstacles.
- The obstacles are already known to the UGV.
- The obstacle density can be generated at three levels:
 - **Low density – 10%**
 - **Medium density – 25%**
 - **High density – 40%**
- The user specifies a **starting point** and a **goal point**.
- The UGV must avoid all obstacles and reach the goal using the **shortest possible path**.

For this purpose, the **A* (A-star) search algorithm** is used.

3. Aim

The main aim of this experiment is:

To design and implement an A* path-finding algorithm that enables a UGV to navigate through a 70 × 70 grid, avoid randomly generated obstacles, and reach a user-defined goal through the shortest possible path.

The performance of the UGV is then evaluated using different **Measures of Effectiveness (MOEs)**.

4. Objectives

The objectives of this experiment are:

1. Create a **70 × 70 grid** representing the battlefield.
 2. Generate obstacles randomly.
 3. Use three different obstacle densities.
 4. Allow the user to specify the **start and goal nodes**.
 5. Implement the **A*** search algorithm.
 6. Find the shortest available path.
 7. Avoid all known obstacles.
 8. Display the path visually.
 9. Measure the algorithm's performance.
 10. Compare the results for different obstacle densities.
-

5. Representation of the Battlefield

The battlefield is represented using a two-dimensional grid.

For example:

```
0 0 0 0 1 0 0
0 1 0 0 1 0 0
0 1 0 0 0 0 0
0 0 0 1 0 1 0
```

Here:

- 0 → Free space
- 1 → Obstacle

The complete battlefield contains:

$$70 \times 70 = 4900$$

cells.

Therefore, the UGV has up to **4900 possible locations** to consider.

6. Start and Goal Nodes

Two locations are selected:

- **Start node:** (2, 2)
- **Goal node:** (67, 67)

The coordinates represent:

(row, column)

The start and goal cells are always kept free of obstacles so that the UGV can begin its journey and attempt to reach the destination.

The program can also be modified so that the user enters these coordinates manually.

7. Obstacle Generation

The obstacles are generated randomly.

Three levels of obstacle density are considered:

Density Percentage Approx. Obstacles		
Low	10%	490
Medium	25%	1225
High	40%	1960

For every cell, a random value is generated. Based on the selected probability, the cell is either made an obstacle or left free.

For example:

- At **10% density**, approximately 10 out of every 100 cells become obstacles.
- At **25% density**, approximately 25 out of every 100 cells become obstacles.
- At **40% density**, approximately 40 out of every 100 cells become obstacles.

Since the obstacles are randomly generated, the exact number and arrangement may slightly differ between runs.

8. Why A* Algorithm?

The A* algorithm is selected because the problem requires the UGV to find a **shortest path** while avoiding obstacles.

A* combines two pieces of information:

1. The cost already travelled.
2. An estimate of the remaining distance.

The main evaluation function is:

$$f(n) = g(n) + h(n)$$

Where:

- $f(n)$ = total estimated cost
- $g(n)$ = actual cost from start to current node
- $h(n)$ = estimated cost from current node to goal

In our grid, every movement costs 1.

9. Heuristic Function

Since the UGV is allowed to move only:

- Up
- Down
- Left
- Right

the **Manhattan distance** is used as the heuristic.

The formula is:

$$h(n) = |x - x_g| + |y - y_g|$$

Where:

- (x, y) = current cell
- (x_g, y_g) = goal cell

Example

Suppose the current position is:

$$(5, 7)$$

and the goal is:

$$(10, 12)$$

Then:

$$\begin{aligned} h(n) &= |5 - 10| + |7 - 12| \\ &= 5 + 5 \\ &= 10 \end{aligned}$$

So the estimated remaining distance is **10 cells**.

10. How A* Works

The algorithm works in a simple sequence.

Step 1: Start from the start node

The UGV begins at:

START = (2, 2)

Step 2: Check neighbouring cells

The algorithm checks four possible directions:

Up
↑
Left ← UGV → Right
↓
Down

Step 3: Ignore obstacles

If a neighbouring cell contains an obstacle, the UGV cannot move there.

Step 4: Calculate the cost

For every valid cell:

$$g(n) = \text{cost travelled so far}$$

Since every movement costs 1:

$$g(\text{new}) = g(\text{current}) + 1$$

Step 5: Calculate heuristic

The Manhattan distance to the goal is calculated:

$$h(n) = |x - x_g| + |y - y_g|$$

Step 6: Calculate total cost

$$f(n) = g(n) + h(n)$$

Step 7: Select the best node

The node having the smallest estimated total cost is selected.

Step 8: Continue until the goal is reached

The process continues until:

- The goal is reached, or
 - There are no more possible nodes to explore.
-

11. Path Reconstruction

Once the goal is reached, the algorithm needs to determine the actual route taken by the UGV.

For this purpose, the program stores the **parent/previous node** of every selected cell.

For example:

```
Start
  ↓
  A
  ↓
  B
  ↓
  C
  ↓
Goal
```

The algorithm works backwards from the goal:

```
Goal → C → B → A → Start
```

Then it reverses the sequence:

```
Start → A → B → C → Goal
```

This gives the final UGV path.

12. Grid Visualization

The generated battlefield is displayed visually.

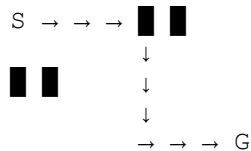
The visualization uses different representations for:

- **White cells** → Free space
- **Black cells** → Obstacles

- **Blue cell** → Start
- **Red cell** → Goal
- **Green cells** → UGV path

This makes it easy to understand how the UGV moves through the battlefield.

A typical result looks conceptually like:



where:

- S = Start
- G = Goal
- ■ = obstacle
- →/↓ = UGV path

13. Measures of Effectiveness (MOEs)

Simply finding a path is not enough. We also need to measure how effectively the algorithm performs.

The following **Measures of Effectiveness** are used.

13.1 Path Length

Path length represents the total number of movements required by the UGV to reach the goal.

If the UGV moves through 120 cells:

$$\text{Path Length} = 120$$

For our grid, every movement costs 1 km.

Therefore:

$$\text{Path Length in km} = \text{Number of movements}$$

A shorter path means less distance travelled.

13.2 Execution Time

Execution time represents how long the A* algorithm takes to find the path.

It is measured using:

```
time.perf_counter()
```

The basic calculation is:

$$\text{Execution Time} = \text{End Time} - \text{Start Time}$$

The time is measured in seconds.

13.3 Nodes Expanded

A node is considered expanded when the algorithm examines it to find possible neighbouring cells.

For example:

```
Start → A → B → C → Goal
```

The algorithm may examine many other cells before reaching the goal.

The number of examined cells is called:

Number of Nodes Expanded

Generally, fewer unnecessary nodes mean that the search was more focused.

13.4 Path Efficiency

Path efficiency compares the shortest theoretical distance with the actual path found.

For our four-directional grid:

$$\text{Direct Distance} = |x_s - x_g| + |y_s - y_g|$$

Then:

$$\text{Path Efficiency} = \frac{\text{Direct Distance}}{\text{Actual Path Length}} \times 100$$

Example

Suppose:

- Direct distance = 130 km
- Actual path = 150 km

Then:

$$\begin{aligned}\text{Efficiency} &= \frac{130}{150} \times 100 \\ &= 86.67\%\end{aligned}$$

A higher percentage means the path is closer to the theoretical shortest distance.

14. Success Rate

Sometimes a path may not exist.

For example, obstacles may completely surround the goal.

Therefore, the experiment records whether the UGV successfully reaches the goal.

For multiple trials:

$$\text{Success Rate} = \frac{\text{Successful Trials}}{\text{Total Trials}} \times 100$$

Example

If 18 out of 20 trials successfully reach the goal:

$$\begin{aligned}\text{Success Rate} &= \frac{18}{20} \times 100 \\ &= 90\%\end{aligned}$$

15. Testing Different Obstacle Densities

The algorithm is tested using three environments:

Low Density – 10%

There are relatively fewer obstacles.

The UGV generally has many possible routes to the destination.

Medium Density – 25%

The number of obstacles increases.

The UGV has fewer free routes and may need to travel around obstacles.

High Density – 40%

The battlefield becomes much more crowded.

The UGV may need to take longer routes, and in some randomly generated maps there may be **no available path**.

16. Why Multiple Trials Are Used

Because obstacles are generated randomly, one run is not enough to make a meaningful comparison.

For example, one random 40% map might happen to have a clear route while another might block the goal completely.

Therefore, the experiment runs:

20 trials per density

That gives:

$$3 \times 20 = 60$$

total experiments.

The average values are then calculated.

This provides a more reliable picture of how obstacle density affects UGV navigation.

17. Experimental Results

The program generates a table similar to:

Density	Avg. Path Length	Avg. Nodes Expanded	Avg. Time	Avg. Efficiency	Success Rate
Low	Calculated by program	Calculated	Calculated	Calculated	Calculated
Medium	Calculated by program	Calculated	Calculated	Calculated	Calculated
High	Calculated by program	Calculated	Calculated	Calculated	Calculated

Note: The exact values change because the obstacles are randomly generated.

Therefore, the actual results should be taken from the Colab output rather than writing fixed values in the report.

18. Expected Observation

As the obstacle density increases, the UGV generally encounters more blocked cells.

Therefore:

- The available routes become fewer.
- The UGV may need to travel around obstacles.
- The path can become longer.
- More nodes may need to be examined.
- Execution time can increase.
- The possibility of finding no path can increase.
- Path efficiency can decrease when detours become necessary.

However, because the maps are randomly generated, these trends are **not guaranteed in every individual trial**.

19. Pseudocode

START

Create 70 × 70 grid

Generate obstacles randomly

Keep START and GOAL cells free

Put START node into priority queue

Set cost of START = 0

```

WHILE priority queue is not empty:

    Select node with lowest f(n)

    IF node is GOAL:
        Reconstruct path
        Return path

    FOR each neighbouring cell:

        IF cell is outside grid:
            Ignore it

        IF cell is an obstacle:
            Ignore it

        Calculate new path cost

        IF new cost is better:
            Update cost
            Store parent node
            Calculate f(n)
            Add node to priority queue

IF no path exists:
    Return failure

END

```

20. Complexity

For a graph implementation of A* using a priority queue, the practical complexity depends on the graph and heuristic. A common upper-bound description is approximately:

$$O((V + E) \log V)$$

where:

- V = number of cells/nodes
- E = connections between neighbouring cells

For a 70×70 grid:

$$V = 4900$$

Each cell has at most four neighbours in our implementation.

The memory requirement is mainly for:

- Open set
- Cost values
- Parent information
- Grid

21. Advantages

1. Finds shortest paths

With the Manhattan heuristic used here, A* can find an optimal path for the four-directional unit-cost grid.

2. Faster than uninformed search in many cases

The heuristic gives the algorithm information about the direction of the goal.

3. Avoids known obstacles

Blocked cells are ignored during navigation.

4. Easy to visualize

The complete UGV route can be displayed on the grid.

5. Flexible

The algorithm can be extended to include different movement costs and more realistic environments.

22. Limitations

The current implementation has some simplifying assumptions:

1. Obstacles are static.
2. All obstacles are known beforehand.
3. Each grid movement has the same cost.
4. The UGV can move only in four directions.
5. The terrain is assumed to be uniform.
6. Random obstacles may sometimes completely block the route.
7. The model does not account for fuel, battery, terrain difficulty, or vehicle dynamics.
8. It does not consider moving obstacles.

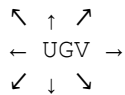
Therefore, this implementation represents a **simplified battlefield navigation model**, rather than a complete real-world UGV navigation system.

23. Possible Future Enhancements

The project can be improved by adding:

Diagonal Movement

Allow:



This can make routes more realistic.

Different Terrain Costs

For example:

Terrain Cost	
Road	1
Grass	2
Mud	4
Mountain	8
Obstacle	∞

The UGV could then choose a route based on both distance and terrain difficulty.

Dynamic Obstacles

Obstacles could move during navigation, requiring the UGV to re-plan its path.

Real Maps

Real geographical or satellite maps could replace the artificial 70×70 grid.

Algorithm Comparison

The same environment can be tested using:

- BFS
- Dijkstra/UCS
- Greedy Best-First Search
- A*

The resulting path length, execution time, and nodes expanded can then be compared.

24. Real-World Applications

The same type of path planning can be used in:

- Autonomous ground vehicles
- Search-and-rescue robots
- Disaster-response robots
- Warehouse robots
- Mine exploration robots
- Agricultural robots
- Delivery robots
- Autonomous navigation systems

The basic idea is always similar:

Know the environment → identify obstacles → search for a safe route → reach the destination efficiently.

25. Observation

The experiment demonstrates that A* can successfully guide a UGV through a grid environment containing known obstacles. The algorithm considers both the distance already travelled and the estimated distance to the goal. When the obstacle density is low, the UGV generally has more possible routes. As obstacle density increases, the available free space decreases and the UGV may need to take longer detours or may sometimes be unable to reach the goal.

Running multiple random trials provides better performance measurements than relying on a single map.

26. Conclusion

In this experiment, a **70 × 70 battlefield grid** was created to simulate the operating environment of an Unmanned Ground Vehicle. Obstacles were randomly generated using low, medium, and high density levels.

The A* search algorithm was implemented to find a path from a user-defined start node to a goal node while avoiding obstacles. The Manhattan distance was used as the heuristic because the UGV was restricted to four-directional movement.

The generated path was displayed graphically, and the performance of the navigation system was measured using **path length, execution time, nodes expanded, path efficiency, and success rate**.

Overall, the experiment shows how an intelligent search algorithm can be used to solve a practical navigation problem in a known environment.
